# JOINS

# SQL JOIN

Before learning **JOIN operations**, we will create a small database that represents a **Student Course System**.
This system will help us understand how tables relate to each other and how JOIN works.

**System Scenario**
We have a small training center that needs to store:

- Students
- Courses
- Student Enrollments in courses

Each student can enroll in multiple courses.

# SQL JOIN

**Creating tables first:**

```
CREATE TABLE students (
    student_id SERIAL PRIMARY KEY,
    student_name VARCHAR(50)
);
CREATE TABLE courses (
    course_id SERIAL PRIMARY KEY,
    course_name VARCHAR(50)
);
CREATE TABLE enrollments (
    enrollment_id SERIAL PRIMARY KEY,
    student_id INT,
    course_id INT,
    FOREIGN KEY (student_id) REFERENCES students(student_id),
    FOREIGN KEY (course_id) REFERENCES courses(course_id)
);
```

# SQL JOIN

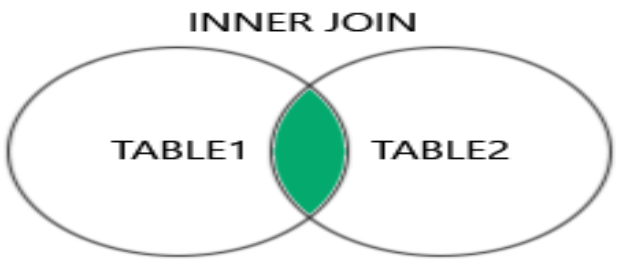
**Then Add data to tables:**

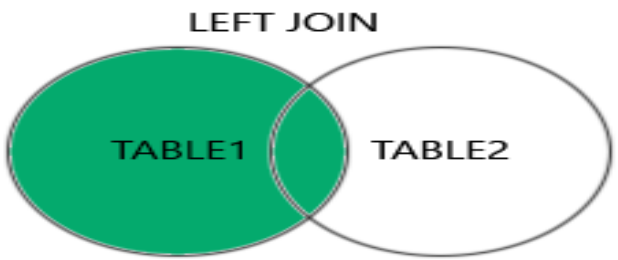
```
INSERT INTO students (student_name) VALUES
('Ali'),('Sara'),('Omar'),('Ahmed');

INSERT INTO courses (course_name) VALUES
('Database'),('JS'),('HTML');

INSERT INTO enrollments (student_id, course_id) VALUES
(1,1),(1,2),(2,1),(3,3);
```

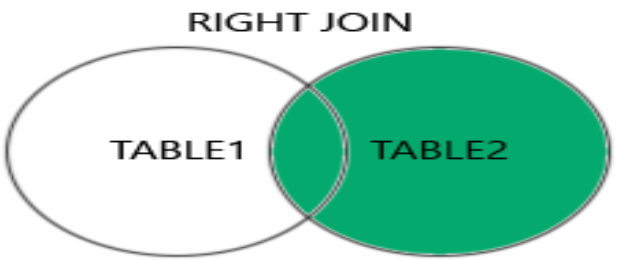
# SQL JOIN

A JOIN clause is used to combine rows from two or more tables, based on a related column between them.
Here are the different types of the JOINs in SQL(Most popular used):

- INNER JOIN: Returns records that have matching values in both tables
- LEFT JOIN: Returns all records from the left table, and the matched records from the right table
- RIGHT JOIN: Returns all records from the right table, and the matched records from the left table



# SQL JOIN

**INNER JOIN - Show all enrolled students and their courses**
**Shows only students who are enrolled in courses.**

**Result : only matching rows**

```
SELECT s.student_name, c.course_name
FROM enrollments e
INNER JOIN students s
ON e.student_id = s.student_id
INNER JOIN courses c
ON e.course_id = c.course_id;
```

**Output:**

| | student_name character varying (50) | course_name character varying (50) |
|---|---|---|
| 1 | Ali | Database |
| 2 | Ali | JS |
| 3 | Sara | Database |
| 4 | Omar | HTML |

# SQL JOIN

## LEFT JOIN - Show all students, and courses if enrolled Ahmed is shown even though he didn't enroll in any course

**Result : Shows all students even if no course.**

```
SELECT s.student_name, c.course_name
FROM students s
LEFT JOIN enrollments e
ON s.student_id = e.student_id
LEFT JOIN courses c
ON e.course_id = c.course_id;
```

**Output:**

| | student_name character varying (50) | course_name character varying (50) |
|---|---|---|
| 1 | Ali | Database |
| 2 | Ali | JS |
| 3 | Sara | Database |
| 4 | Omar | HTML |
| 5 | Ahmed | [null] |

# SQL JOIN

**RIGHT JOIN – Show all courses, and the students enrolled, even though no student is enrolled in it.**

**Result : Returns all courses even if no students.**

```
SELECT s.student_name, c.course_name
FROM students s
RIGHT JOIN enrollments e
ON s.student_id = e.student_id
RIGHT JOIN courses c
ON e.course_id = c.course_id;
```

**Output:**

| | student_name character varying (50) | course_name character varying (50) |
|---|---|---|
| 1 | Ali | Database |
| 2 | Ali | JS |
| 3 | Sara | Database |
| 4 | Omar | HTML |

# SQL JOIN

Will take some examples of joins with aggregation functions

## 1. Count courses per student

```
SELECT s.student_name, COUNT(e.course_id) AS total_courses
FROM students s
LEFT JOIN enrollments e
ON s.student_id = e.student_id
GROUP BY s.student_name;
```

**Output:**

| | student_name character varying (50) | total_courses bigint |
|---|---|---|
| 1 | Ali | 2 |
| 2 | Sara | 1 |
| 3 | Ahmed | 0 |
| 4 | Omar | 1 |

# SQL JOIN

Will take some examples of joins with aggregation functions

## 2. Number of students in each course

```
SELECT c.course_name, COUNT(e.student_id) AS total_students
FROM courses c
LEFT JOIN enrollments e
ON c.course_id = e.course_id
GROUP BY c.course_name;
```

**Output:**

| | course_name character varying (50) | total_students bigint |
|---|---|---|
| 1 | JS | 1 |
| 2 | Database | 2 |
| 3 | HTML | 1 |

# SQL JOIN

Will take some examples of joins with aggregation functions

## 3. Show students with number of courses sorted by most courses

```
SELECT s.student_name, COUNT(e.course_id) AS total_courses
FROM students s
LEFT JOIN enrollments e
ON s.student_id = e.student_id
GROUP BY s.student_name
ORDER BY total_courses DESC;
```

**Output:**

| | student_name character varying (50) | total_courses bigint |
|---|---|---|
| 1 | Ali | 2 |
| 2 | Sara | 1 |
| 3 | Omar | 1 |
| 4 | Ahmed | 0 |

# SQL JOIN

Will take some examples of joins with aggregation functions

4. Show most popular courses

```
SELECT c.course_name, COUNT(e.student_id) AS total_students
FROM courses c
LEFT JOIN enrollments e
ON c.course_id = e.course_id
GROUP BY c.course_name
ORDER BY total_students DESC;
```

**Output:**

| | course_name character varying (50) | total_students bigint |
|---|---|---|
| 1 | Database | 2 |
| 2 | JS | 1 |
| 3 | HTML | 1 |

# SQL JOIN

A JOIN clause is used to combine rows from two or more tables, based on a related column between them.
Here are the different types of the JOINs in SQL:

- FULL JOIN : The FULL JOIN keyword selects ALL records from both tables, even if there is not a match. For rows with a match the values from both tables are available, if there is not a match the empty fields will get the value NULL.
- CROSS JOIN : The CROSS JOIN keyword returns the Cartesian product of two or more tables (combines every row from the first table with every row from the second table).
- Self join:A self join means joining a table with itself.
- Lateral join :LATERAL allows a subquery to use columns from a previous table.
- Natural join: JOIN automatically joins tables based on columns that have the same name in both tables.

# SQL JOIN

FULL JOIN – All records from both tables are returned.

**Result :** Returns **everything from both tables.**

```
SELECT s.student_name, c.course_name
FROM students s
FULL JOIN enrollments e
ON s.student_id = e.student_id
FULL JOIN courses c
ON e.course_id = c.course_id;
```

**Output:**

| | student_name character varying (50) | course_name character varying (50) |
|---|---|---|
| 1 | Ali | Database |
| 2 | Ali | JS |
| 3 | Sara | Database |
| 4 | Omar | HTML |
| 5 | Ahmed | [null] |

# SQL JOIN

CROSS JOIN - if the first table has 100 rows, and the second table has 500 rows, the result set will be 100x500 rows. Be aware that CROSS JOIN can potentially return **very large result-sets**!

**Result :** 4 students × 3 courses = **12 rows**

```
SELECT s.student_name, c.course_name
FROM students s
CROSS JOIN courses c;
```

**Output:**

| | student_name character varying (50) | course_name character varying (50) |
|---|---|---|
| 1 | Ali | Database |
| 2 | Ali | JS |
| 3 | Sara | Database |
| 4 | Omar | HTML |
| 5 | Ahmed | [null] |

# SQL JOIN

A self join is a regular join, but the table is joined with itself.

Example: If students table stores **mentor_id** referencing another student.

```
SELECT s1.name AS student, s2.name AS mentor
FROM students s1
JOIN students s2
ON s1.mentor_id = s2.student_id;
```

```
SELECT e1.name AS manager, e2.name AS emp_name
FROM employees e1
JOIN employees e2
ON e1.id = e2.manager_id;
```

| manager character varying (50) | emp_name character varying (50) |
|---|---|
| Ahmed | Mohamed |
| Ahmed | Mona |
| Sara | Ali |
| Sara | mohAMed |

# SQL JOIN

## JOIN LATERAL

LATERAL allows a subquery in the FROM clause to **use columns from previous tables.**

```sql
SELECT s.student_name, c.course_name
FROM students s
JOIN LATERAL (
    SELECT course_id
    FROM enrollments e
    WHERE e.student_id = s.student_id
) e ON TRUE
JOIN courses c ON c.course_id = e.course_id;
```

| student_name character varying (50) | course_name character varying (50) |
|---|---|
| Ali | Database |
| Ali | JS |
| Sara | Database |
| Omar | HTML |

**Note: This query produces the same result as a normal join**

# SQL JOIN

## NATURAL JOIN

NATURAL JOIN automatically joins tables based on **columns that have the same name in both tables.**

```
SELECT *
FROM students
NATURAL JOIN enrollments;
```

| student_id integer | student_name character varying (50) | enrollment_id integer | course_id integer |
|---|---|---|---|
| 1 | Ali | 1 | 1 |
| 1 | Ali | 2 | 2 |
| 2 | Sara | 3 | 1 |
| 3 | Omar | 4 | 3 |

If both tables contain `student_id`, SQL automatically joins using it.
Not commonly used in production because it can join on unexpected columns.

# SQL JOIN

## JOIN with Subqueries

Joining a table with the **result of a subquery**.

Get student's name and the total number of courses they are enrolled in

```sql
SELECT s.student_name, sub.total_courses
FROM students s
JOIN (
    SELECT student_id, COUNT(*) AS total_courses
    FROM enrollments
    GROUP BY student_id
) sub
ON s.student_id = sub.student_id;
```

| student_name character varying (50) | total_courses bigint |
|---|---|
| Ali | 2 |
| Sara | 1 |
| Omar | 1 |

# SQL JOIN

## JOIN with Subqueries

Task : Show students who take **more than 1 course**

# SQL JOIN

## JOIN with Subqueries

Task : Show students who take **more than 1 course**

```sql
SELECT s.student_name, sub.total_courses
FROM students s
JOIN (
    SELECT student_id, COUNT(*) AS total_courses
    FROM enrollments
    GROUP BY student_id
) sub
ON s.student_id = sub.student_id
WHERE sub.total_courses > 1;
```

# SQL JOIN

## JOIN with Subqueries

```
CREATE TABLE parents (
    id SERIAL PRIMARY KEY,
    name TEXT NOT NULL
);
```

```
INSERT INTO parents (name) VALUES('Ahmed Hassan'),('Fatima Ali'),('Mohamed Said'),('Sara Ibrahim'),('Omar Khaled’);
insert into students(student_name) values('Karim’);
select * from students;
INSERT INTO student_parents (student_id, parent_id) VALUES(1,1),(1,2),(2,1),(3,3),(4,4),(4,5),(5,3);
```

```
CREATE TABLE student_parents (    student_id INT REFERENCES students(student_id) ON DELETE CASCADE,    parent_id INT REFERENCES parents(id) ON DELETE CASCADE,    PRIMARY KEY (student_id, parent_id));
```

# SQL JOIN

## JOIN with more than 2 tables

List students along with their parents and the courses they are enrolled in, ordered by student name.

```sql
SELECT
    s.student_name AS student,
    p.name AS parent,
    c.course_name AS course
FROM students s
JOIN student_parents sp ON s.student_id = sp.student_id
JOIN parents p ON p.id = sp.parent_id
JOIN enrollments e ON e.student_id = s.student_id
JOIN courses c ON c.course_id = e.course_id
ORDER BY student;
```

| | student character varying (50) | parent text | course character varying (50) |
|---|---|---|---|
| 1 | Ali | Ahmed Hass... | JS |
| 2 | Ali | Ahmed Hass... | Database |
| 3 | Ali | Fatima Ali | JS |
| 4 | Ali | Fatima Ali | Database |
| 5 | Omar | Mohamed Sa... | HTML |
| 6 | Sara | Ahmed Hass... | Database |

# SQL JOIN

JOIN with more than 2 tables

**Task : Show Parents And Their Children**

# SQL JOIN

JOIN with more than 2 tables

**Task : Show Parents And Their Children**

```sql
SELECT
    p.name AS parent_name,
    s.student_name
FROM parents p
JOIN student_parents sp
ON p.id = sp.parent_id
JOIN students s
ON s.student_id = sp.student_id
ORDER BY parent_name;
```

# SQL JOIN

## USING (To Reduce Typing)

The USING clause is a shorthand that allows you to take advantage of the specific situation where both sides of the join use the same name for the joining column(s).

```
SELECT *
FROM students
JOIN enrollments USING (student_id);
```

| student_id integer | student_name character varying (50) | enrollment_id integer | course_id integer |
|---|---|---|---|
| 1 | Ali | 1 | 1 |
| 1 | Ali | 2 | 2 |
| 2 | Sara | 3 | 1 |
| 3 | Omar | 4 | 3 |

# SQL JOIN

## USING (To Reduce Typing)

Compare between On & Using

```
SELECT *
FROM students
JOIN enrollments USING (student_id);
```

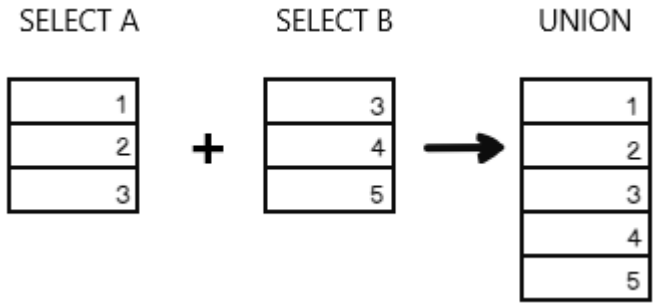
```
SELECT *
FROM students
JOIN enrollments
ON students.student_id = enrollments.student_id;
```

# Operators

## UNION

UNION effectively appends the result of query2 to the result of query1 (although there is no guarantee that this is the order in which the rows are actually returned).

- Furthermore, it eliminates duplicate rows from its result, in the same way as DISTINCT, unless UNION ALL is used.




```
SELECT student_id FROM enrollments
UNION
SELECT student_id FROM students;
```

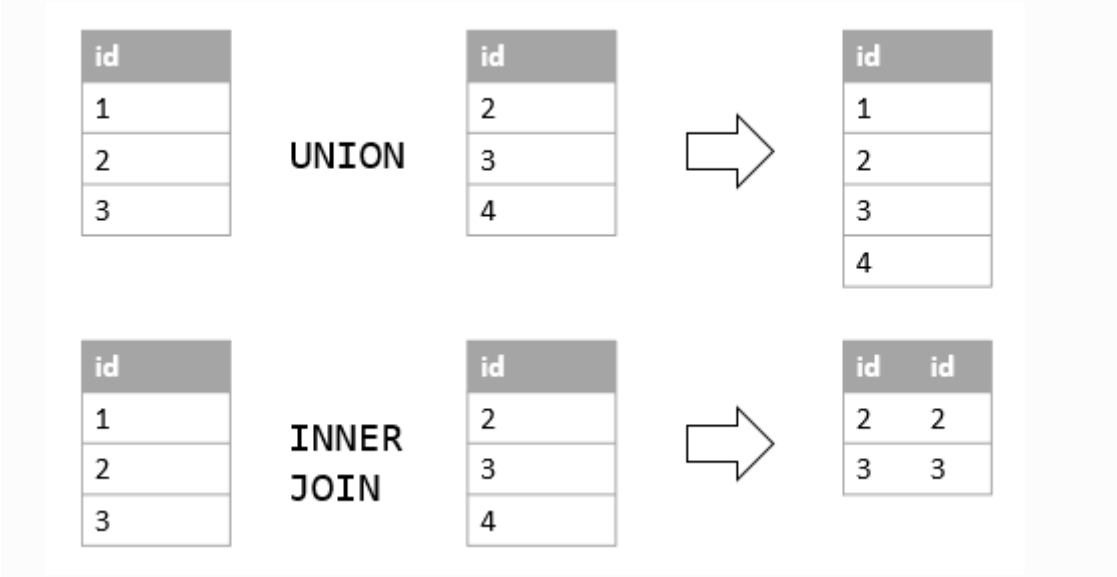
| student_id integer |
|---|
| 2 |
| 4 |
| 3 |
| 1 |

Result contains **unique student IDs only**.

# Operators

## UNION

The difference between **JOIN and UNION** in SQL is mainly **how they combine data.**
A good way to explain it to students is:
JOIN → combine columns from different tables
UNION → combine rows from different queries



# Operators

## UNION ALL

UNION ALL combines results but **keeps duplicates.**

```
SELECT student_id FROM enrollments
UNION ALL
SELECT student_id FROM students;
```

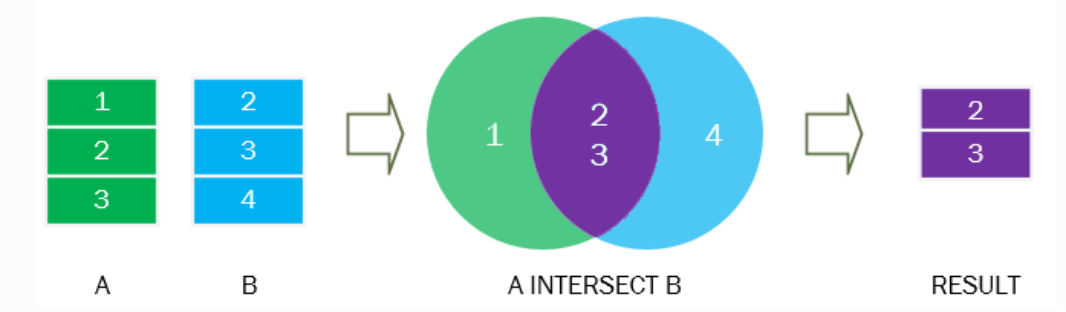
| student_id integer |
|---|
| 1 |
| 1 |
| 2 |
| 3 |
| 1 |
| 2 |

l rows: 8 | Query complete 00:00:00.079

# Operators

## INTERSECT

- INTERSECT returns all rows that are both in the result of query1 and in the result of query2.
- Duplicate rows are eliminated unless INTERSECT ALL is used.



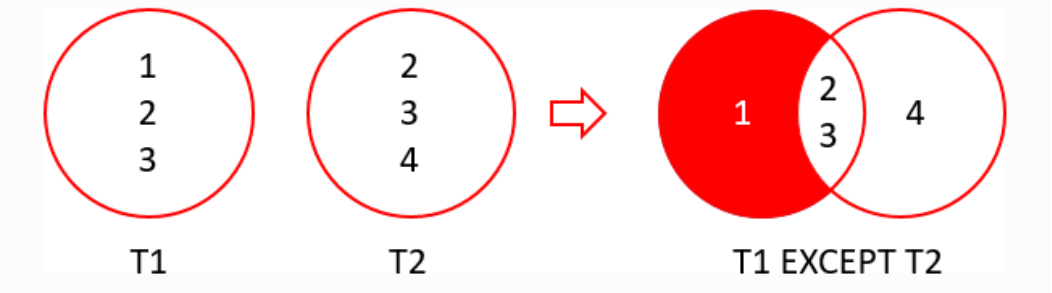

```
SELECT student_id FROM students
INTERSECT
SELECT student_id FROM enrollments;
```

| student_id integer |
|---|
| 3 |
| 2 |
| 1 |

# Operators

## EXCEPT

- EXCEPT returns all rows that are in the result of query1 but not in the result of query2. (This is sometimes called the difference between two queries.)
- Duplicates are eliminated unless EXCEPT ALL is used.




```
SELECT student_id FROM students
EXCEPT
SELECT student_id FROM enrollments;
```

| | student_id integer |
|---|---|
| 1 | 4 |

Result: Students **not enrolled in any course.**

# CTE – Common Table Expression

## WITH

A CTE (WITH clause) is a named temporary result set. Like a temporary view for one query . Makes complex queries readable and maintainable. More powerful than subqueries.

### Basic CTE

Count how many high-earning employees (salary > 8000) exist in each department

```
WITH high_earners AS (SELECT * FROM employees WHERE salary > 8000)
SELECT dep_id, COUNT(*)
FROM high_earners GROUP BY dep_id; -- Use the CTE like a table.
```

| dep_id integer | count bigint |
|---|---|
| 2 | 2 |
| 3 | 4 |

# CTE – Common Table Expression

## WITH

**Multiple CTEs**

**Find all departments where the average employee salary is above 7000 — and show their names**

```sql
WITH dept_avg AS (
    SELECT dep_id, AVG(salary) AS avg_sal
    FROM employees
    GROUP BY dep_id
),
top_depts AS (
    SELECT *
    FROM dept_avg
    WHERE avg_sal > 7000
)
SELECT *
FROM top_depts
JOIN departments
ON top_depts.dep_id = departments.id;
```

| dep_id integer | avg_sal numeric | id integer | department_name character varying (50) |
|---|---|---|---|
| 2 | 8820.0000000000000000 | 2 | HR |
| 3 | 8337.5000000000000000 | 3 | Finance |
| 4 | 7500.0000000000000000 | 4 | Marketing |

# Transactions in PostgreSQL

These commands are part of SQL Transaction Control. **A transaction is a group of SQL operations executed as a single unit**. They ensure data consistency.

Example : Bank transfer

1. START TRANSACTION

`START TRANSACTION` begins a new transaction. After this, SQL statements are executed but **not permanently saved** until you use `COMMIT`.

# Transactions in PostgreSQL

1. START TRANSACTION

```
START TRANSACTION;

INSERT INTO employees(name,salary,dep_id)
VALUES ('Try Transaction', 1000,3);
```

| | id [PK] integer | name character varying (50) | age integer | salary numeric | dep_id integer | manager_id integer |
|---|---|---|---|---|---|---|
| 11 | 17 | Name | [null] | 14000 | 3 | [null] |
| 12 | 3 | Mohamed | 35 | 7000 | 2 | 1 |
| 13 | 4 | Mona | 32 | 9000 | 3 | 1 |
| 14 | 5 | Ali | 26 | 5000 | 2 | 2 |
| 15 | 11 | mohAMed | 32 | 7100 | 2 | 2 |
| 16 | 1 | Ahmed | 30 | 8000 | 1 | [null] |
| 17 | 2 | Sara | 28 | 6000 | 1 | [null] |
| 18 | 18 | Try Transaction | [null] | 1000 | 3 | [null] |

At this point the change is **temporary**.

# Transactions in PostgreSQL

2. COMMIT

`COMMIT` **permanently saves** all changes made in the current transaction to the database.

```
START TRANSACTION;

INSERT INTO employees(name,salary,dep_id)
VALUES ('Try Transaction', 1000,3);

COMMIT;
```

Data Output | Messages | Notifications

```
WARNING:  there is already a transaction in progress
COMMIT

Query returned successfully in 91 msec.
```

| | | | | | | |
|---|---|---|---|---|---|---|
| 18 | 18 | Try Transaction | [null] | 1000 | 3 | [null] |
| 19 | 19 | Try Transaction | [null] | 1000 | 3 | [null] |

After `COMMIT`, the data **cannot be undone with ROLLBACK**.

# Transactions in PostgreSQL

3. ROLLBACK

ROLLBACK **undoes all changes** made in the current transaction and returns the database to the previous

```
START TRANSACTION;

DELETE FROM enrollments
WHERE student_id = 1;

ROLLBACK;
```

| enrollment_id [PK] integer | student_id integer | course_id integer |
|---|---|---|
| 1 | 1 | 1 |
| 2 | 1 | 2 |
| 3 | 2 | 1 |
| 4 | 3 | 3 |

The delete operation is **cancelled** and the data remains unchanged.

# Json In PQL

In **PostgreSQL**, json and jsonb are two data types used to store JSON data, but they have important differences that affect storage, performance, and functionality. Let's go step by step.

**1. `json` type**

•Stores **exactly the JSON text** as input.
•Preserves **whitespace and key order**.
•Parsing is done **at query time**, so operations like searching or filtering require parsing each time.
•**Pros**:

- Keeps the original format intact.
- Slightly faster for simple storage and retrieval.

•**Cons**:

- Slower for querying and indexing.
- Cannot directly perform advanced operations efficiently (like indexing nested keys).

# Json In PQL

Json

```
CREATE TABLE test_json (
    data json
);

INSERT INTO test_json VALUES ('{"name": "Alice", "age": 30}');
```

```
SELECT data->>'name' FROM test_json;  -- returns 'Alice'
```

# Json In PQL

**2. jsonb type**
Stands for **JSON Binary**.
Stores JSON in a **binary format**, not text.
Keys are **sorted**, duplicates are removed, and whitespace is ignored.
Supports **indexing** (GIN, B-Tree for some operators), making queries **much faster**.
**Pros**:

- Fast read and write for complex queries.
- Supports indexing and advanced operators (e.g., containment, existence checks).
- Ideal for structured JSON storage and frequent querying.

**Cons**:

- Slightly slower for simple insertions because input must be converted to binary.
- Loses original formatting (whitespace, key order).

# Json In PQL

**2. jsonb type**

```
CREATE TABLE test_jsonb (
    data jsonb
);

INSERT INTO test_jsonb VALUES ('{"name": "Alice", "age": 30}');
```

```
-- Check if 'age' exists
SELECT * FROM test_jsonb WHERE data ? 'age';

-- Containment query
SELECT * FROM test_jsonb WHERE data @> '{"name": "Alice"}';
```

# Json In PQL

**2. jsonb type**

| Feature | json | jsonb |
|---|---|---|
| Storage format | Text | Binary |
| Key order preserved | ✅ Yes | ❌ No (sorted) |
| Duplicate keys | ✅ Allowed | ❌ Removed |
| Query performance | Slower (parsed at query) | Faster (binary + indexable) |
| Indexing support | ❌ No | ✅ Yes (GIN, B-Tree for some ops) |
| Ideal use case | Storing raw JSON | Query-heavy JSON data |

## Advance Queries

For each parent, get a JSON list of their children — and for each child, include the courses they're enrolled in

```sql
SELECT
    p.name AS parent,
    json_agg(DISTINCT jsonb_build_object(
        'student', s.student_name,
        'courses', (
            SELECT json_agg(c.course_name)
            FROM enrollments e
            JOIN courses c ON c.course_id = e.course_id
            WHERE e.student_id = s.student_id
        )
    )) AS children_info
FROM parents p
JOIN student_parents sp ON sp.parent_id = p.id
JOIN students s ON s.student_id = sp.student_id
GROUP BY p.id;
```

| parent<br>text | children_info<br>json |
|---|---|
| Ahmed Ha... | [{"courses": ["Database"], "student": "Sara"}, {"courses": ["Database", "JS"], "student": "Ali"}] |
| Fatima Ali | [{"courses": ["Database", "JS"], "student": "Ali"}] |
| Mohamed ... | [{"courses": null, "student": "Karim"}, {"courses": ["HTML"], "student": "Omar"}] |
| Sara Ibrahi... | [{"courses": null, "student": "Ahmed"}] |
| Omar Khal... | [{"courses": null, "student": "Ahmed"}] |

## Advance Queries

For each course, how many students are enrolled — and how many of those students have siblings also in the same course?

```sql
WITH course_siblings AS (
    SELECT
        c.course_id AS course_id,
        s.student_id AS student_id,
        COUNT(DISTINCT sp2.student_id) FILTER (WHERE sp1.student_id <> sp2.student_id) AS sibling_count
    FROM enrollments e
    JOIN students s ON s.student_id = e.student_id
    JOIN courses c ON c.course_id = e.course_id
    JOIN student_parents sp1 ON sp1.student_id = s.student_id
    JOIN student_parents sp2 ON sp1.parent_id = sp2.parent_id
    GROUP BY c.course_id, s.student_id
)
SELECT
    c.course_name,
    COUNT(DISTINCT cs.student_id) AS student_count,
    SUM(cs.sibling_count) AS total_sibling_pairs
FROM courses c
JOIN course_siblings cs ON cs.course_id = c.course_id
GROUP BY c.course_id
ORDER BY student_count DESC;
```

| course_name character varying (50) | student_count bigint | total_sibling_pairs numeric |
|---|---|---|
| Database | 2 | 2 |
| JS | 1 | 1 |
| HTML | 1 | 1 |

## Advance Queries

Find all groups of siblings — students who share the same parent

```
WITH sibling_groups AS (
    SELECT
        sp1.parent_id,
        array_agg(DISTINCT s1.student_name) AS children
    FROM student_parents sp1
    JOIN students s1 ON s1.student_id = sp1.student_id
    GROUP BY sp1.parent_id
)
SELECT DISTINCT children
FROM sibling_groups
WHERE array_length(children,1) > 1;
```

| | children character varying[] |
|---|---|
| 1 | {Ali,Sara} |
| 2 | {Karim,Omar} |

# Thank you